

Lecture 13: Data as matrices

Jamie Haddock

Table of contents

1	From matrix to insight	1
1.1	Tables as matrices	1
1.2	Graphs as matrices	1
1.3	Adjacency matrices	2
1.4	Images as matrices	3

1 From matrix to insight

Any two-dimensional array of numbers may be interpreted as a matrix, although this is certainly not the only representation of such data.

Matrix-related mathematical and computational tools are universally applicable and often applied with great success.

1.1 Tables as matrices

Tables are used to represent variation of a quantity with respect to two variables. These variables may be encoded as the rows and columns of a matrix.

Example: A *corpus* is a collection of text documents. A *term-document matrix* has one column for each document and one row for each unique term appearing in the corpus. The (i, j) entry of the matrix is the number of times term i appears in document j . Matrix decomposition methods are often applied in this application. The SVD leads to an important method known as *latent semantic analysis*.

Example: Each vote cast in the U. S. Congress is [available for download](#). We can put members of Congress along the columns of a matrix and bills along the rows, recording a number that codes for “yea,” “nay,” “none,” etc. The SVD reveals an analysis of the partisanship and cooperation of individual members.

Example: In 2006, Netflix started an open competition for a \$1 million prize. They provided a 17,770 movie-by-480,189 user matrix dataset, with each entry a 1-5 star rating. The object was to predict a user’s ratings for movies they had not rated. This is known as a *matrix completion problem*. Teams nearly immediately improved Netflix’s approach, and in 2009 the prize was awarded for 10% improvement using techniques related to NMF.

1.2 Graphs as matrices

We saw briefly previously that graphs lead us to (sparse) matrices.

Definition: Graphs and adjacency matrices

A **graph** or *network* consists of a set V of **nodes** and a set E of **edges**, each of which is an ordered pair of nodes. The graph is **undirected** if for every edge (v_i, v_j) , the pair (v_j, v_i) is also an edge; otherwise the graph is **directed**.

The **adjacency matrix** of a graph with n nodes V and edge set E is the $n \times n$ matrix whose elements are

$$A_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in E \text{ (i.e., there is an edge from node } i \text{ to node } j), \\ 0 & \text{otherwise.} \end{cases}$$

Applications:

- social networks
- airline routes
- power grids
- sports teams
- web pages

1.3 Adjacency matrices

Here are some elementary results about adjacency matrices.

Theorem:

For any graph with adjacency matrix $\mathbf{A}$,

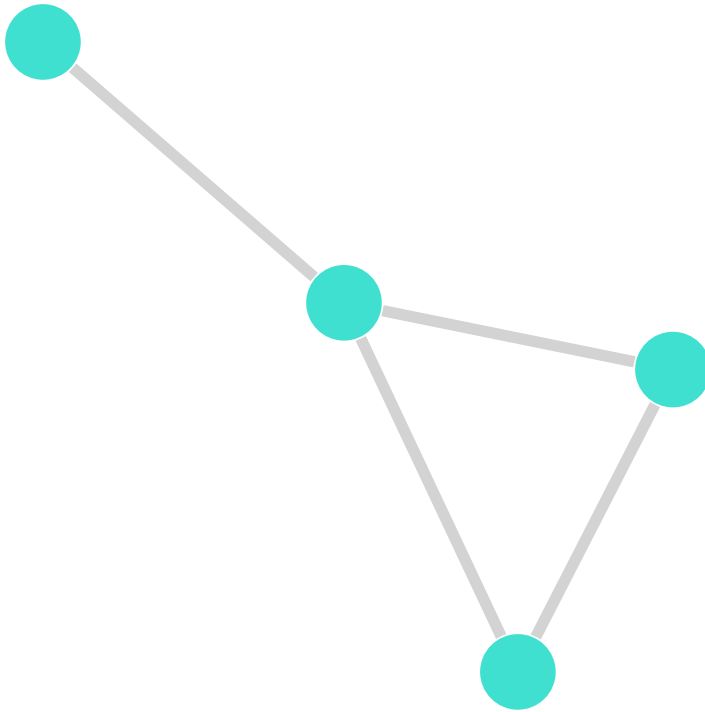
1. The graph is undirected if and only if $\mathbf{A}$ is symmetric, and
2. For any positive integer k , the (i, j) element of $\mathbf{A}^k$ is the number of ways to walk from node i to node j by following along exactly k edges.

Here we create an adjacency matrix for a graph on four nodes.

```
Project No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Manifest No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Project No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Manifest No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Project No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Manifest No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Project No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
Manifest No packages added to or removed from ~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/

%%julia
using Graphs
using GraphPlot

A = [0 1 1 1; 1 0 0 1; 1 0 0 0; 1 1 0 0]
G = Graph(A)
gplot(G)
```



Here are the counts of all walks of length 3 in the graph:

```
%%julia
A^2
```

```
4×4 Matrix{Int64}:
 3  1  0  1
 1  2  1  1
 0  1  1  1
 1  1  1  2
```

The representation of a graph by its adjacency matrix opens up the possibility of many kinds of analysis of the graph. One might ask whether the nodes admit a natural partition into clusters, for example. Or one might ask to rank the nodes in order of importance to the network as determined by some objective criteria—an application made famous by Google’s PageRank algorithm.

All of these are mathematically stated as problems about matrices.

1.4 Images as matrices

Computers typically represent images as rectangular arrays of pixels, each of which is colored according to numerical values for red (R), green (G), and blue (B) components of white light. Most often, these are given as integers in the range from zero (no color) to 255 (full color). Thus, an image that is m -by- n pixels can be stored as an m -by- n -by-3 array of integer values. In Julia, we can work with an $m \times n$ matrix of 3-vectors (or tensor).

```
Resolving package versions...
Updating `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/Project.toml`
[5e47fb64] + TestImages v1.9.0
Updating `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/Manifest.toml`
[88034a9c] + StringDistances v0.11.3
```

```
[5e47fb64] + TestImages v1.9.0
Info Packages marked with  have new versions available but compatibility constraints restrict t
Precompiling packages...
 0.6 s   StringDistances
 1.3 s   TestImages
2 dependencies successfully precompiled in 3 seconds. 112 already precompiled.
```

```
%%julia
img = testimage("mandrill")
```



```
%%julia
size(img)
```

```
(512, 512)
```

```
%%julia
img[100,10]
```



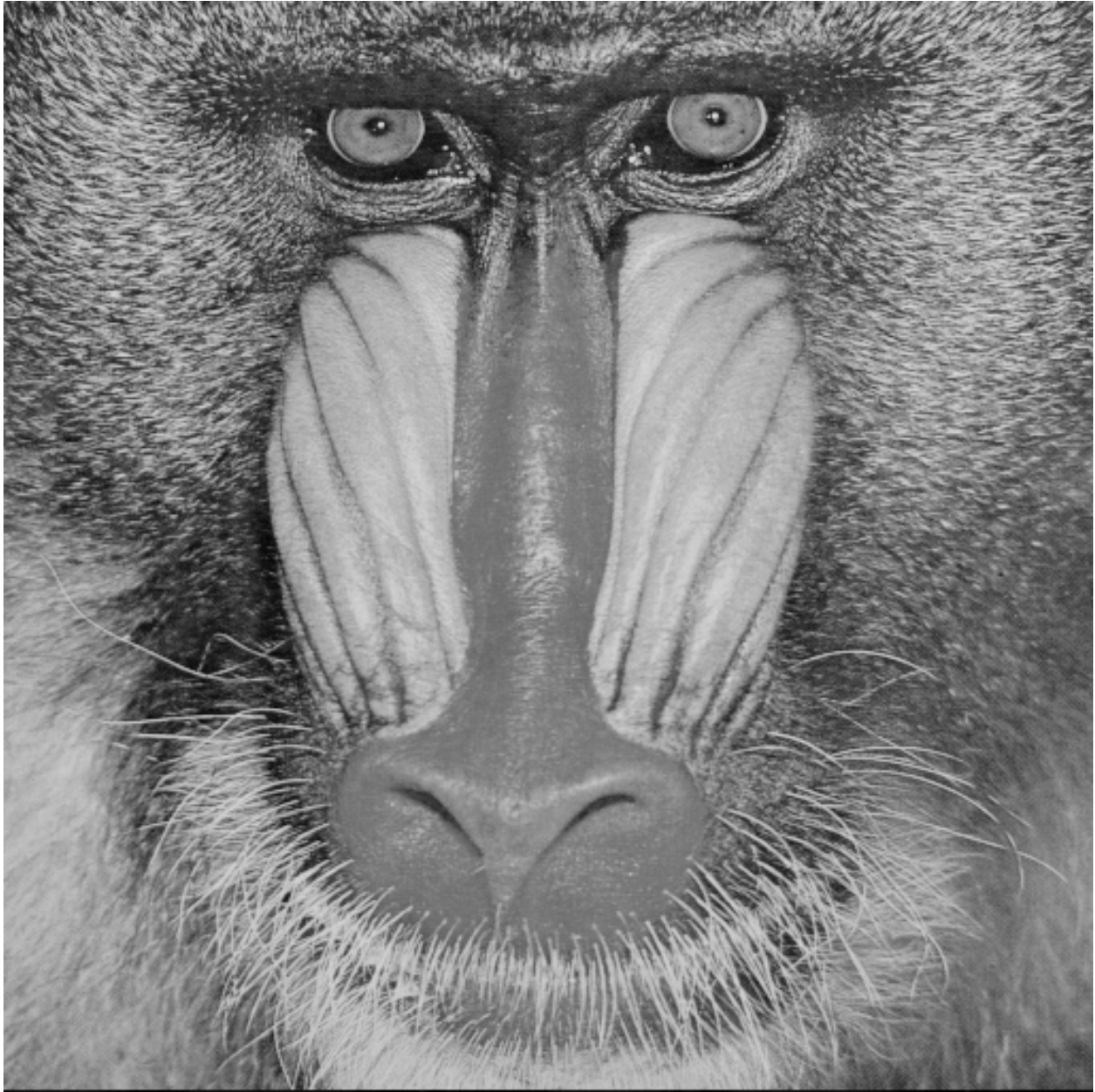
```
%%julia
eltype(img) # find out the type of the elements of any array
```

```
RGB{N0f8}
```

```
%%julia
R,G,B = red.(img),green.(img),blue.(img); # extract color channel matrices
@show minB,maxB = extrema(B);
```

```
(minB, maxB) = extrema(B) = (0.0N0f8, 1.0N0f8)
```

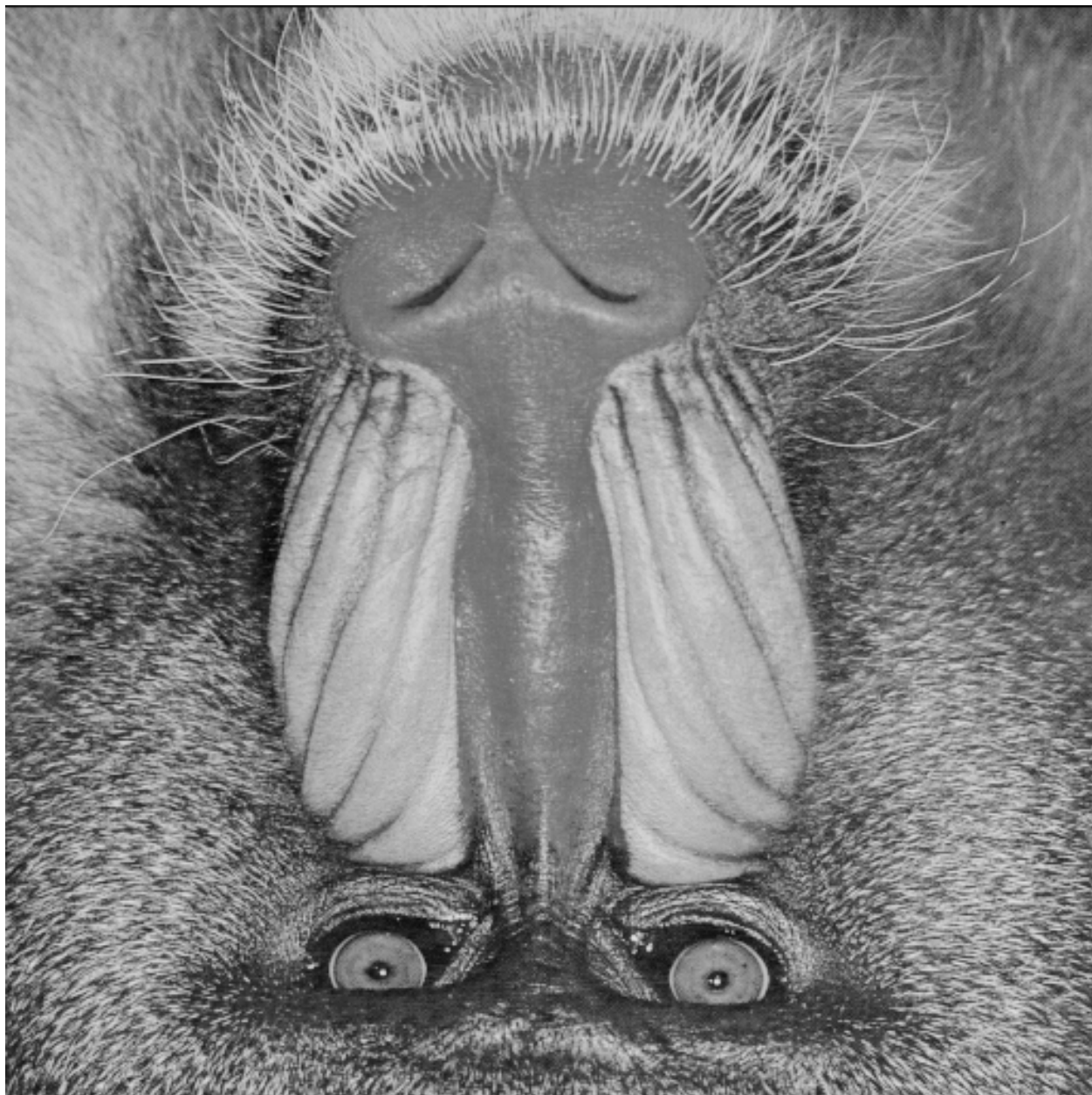
```
%%julia
Gray.(img) # convert to grayscale
```



```
%%julia
A = Float64.(Gray.(img)) # we have to let Julia know to interpret this numerically
A[1:4,1:5]
```

```
4×5 Matrix{Float64}:
 0.568627  0.219608  0.192157  0.34902  0.537255
 0.454902  0.396078  0.156863  0.262745  0.352941
 0.301961  0.447059  0.180392  0.180392  0.384314
 0.278431  0.533333  0.372549  0.188235  0.32549
```

```
%%julia
Gray.(reverse(A,dims=1)) # Gray allows us to interpret the floating
                        # point matrix as a grayscale image
```



Representation of an image as a matrix allows us to describe some common image operations in terms of linear algebra. For example, we will use the SVD to compress the image data, and we will use Krylov methods to apply and remove blurring effects.